

STUDENT RESULT MANAGEMENT SYSTEM

IT23431- MONGODB ESSENTIALS

Submitted by

HARISH DJ – 241001077

JASHWANTH J – 241001092

GEEVAN G. V – 241001061

OF

BACHELOR OF INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project titled “Student Result Management System” is the bonafide work of “ **HARISH DJ (241001077),JASHWANTH J(241000192),GEEVAN GV(241001061),**” who carried out the project work under my supervision.

SIGNATURE

Dr.Valarmathie

HEAD OF THE DEPARTMENT

Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai – 602105

SIGNATURE

Mrs.Usha S

COURSE INCHARGE

Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai - 602105

This project is submitted for IT23431- MONGODB ESSENTIALS
held on _____

INTERNAL EXAMINAR

EXTERNAL EXAMINAR

TABLE OF CONTENTS

S.NO	TOPIC	PAGE NO
1	Abstract	1
2	Introduction	1
3	Purpose	1
4	Scope of the project	2
5	Software requirement specification	2
6	Use case diagram	3
7	Entity relationship diagram	4
8	Data flow diagram	5
9	Module description	5
10	Project functionality	5
11	Implementation	11
12	Conclusion	11

STUDENT RESULT MANAGEMENT SYSTEM

1.ABSTRACT

The Student Result Management System is a web-based application developed to manage student academic records efficiently. The system uses MongoDB as the primary database and supports operations such as adding, updating, viewing, and deleting student information. The application is developed using HTML, CSS, and JavaScript for the frontend, and Node.js with Express.js for the backend. MongoDB's flexible document-oriented model allows storing student data in JSON-like documents, making the system scalable and easy to manage.

2.INTRODUCTION

Managing student results manually is time-consuming and prone to errors. The Student Result Management System is designed to digitize the process of storing and managing student academic records. It provides a centralized platform for administrators to handle student data effectively.

3.PURPOSE

- Develop a system for managing student results digitally
- Reduce manual errors
- Provide quick access to data
- Demonstrate CRUD operations
- Showcase full-stack development

4.SCOPE OF THE PROJECT

- Manage student details
- Record subject-wise marks
- Update and delete records
- Display results

5.SOFTWARE REQUIREMENT SPECIFICATION

Functional Requirements:

- Add, view, update, delete student data

Non-Functional Requirements:

- User-friendly
- Scalable
- Fast

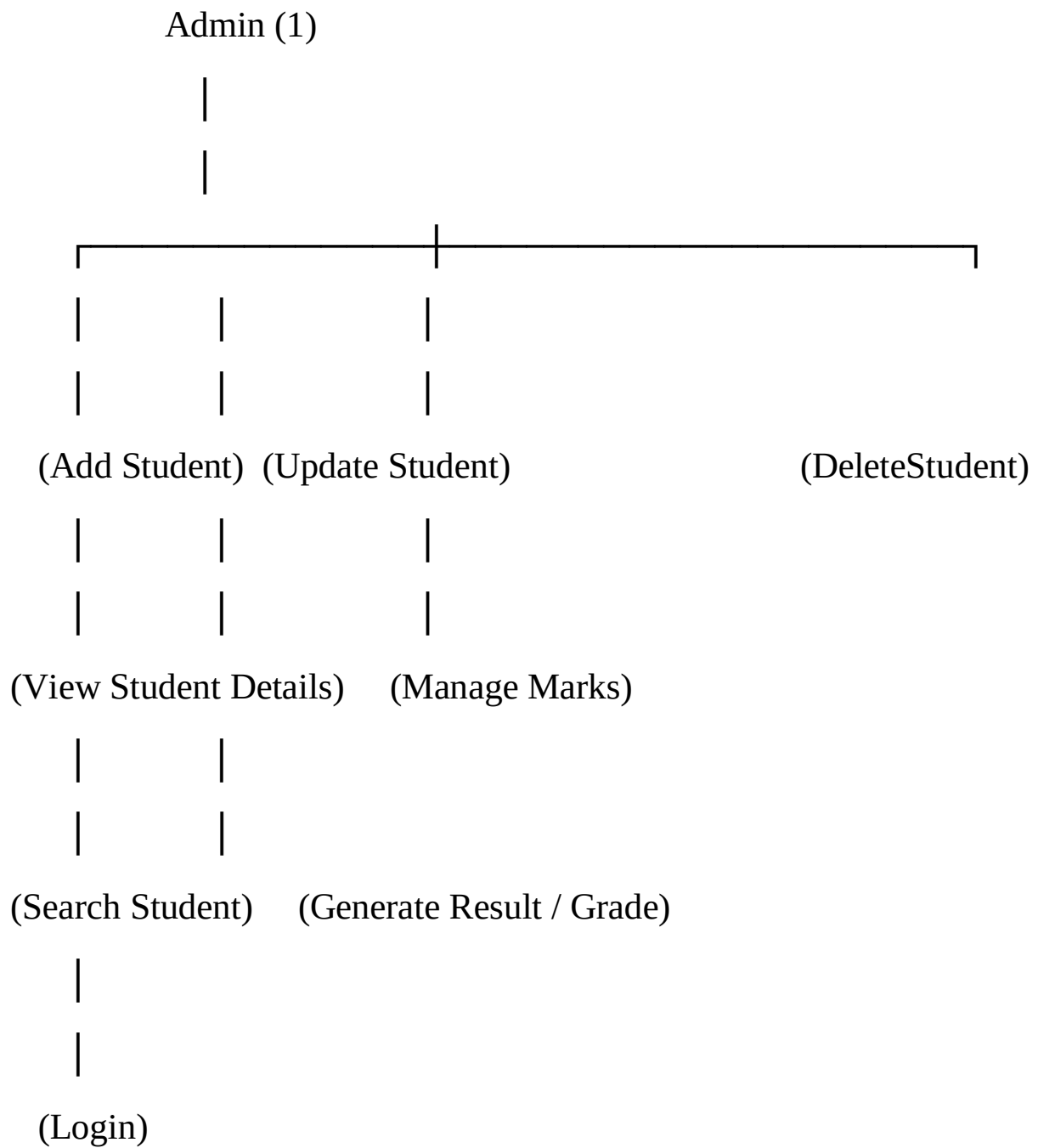
Hardware Requirements:

- 4GB RAM system

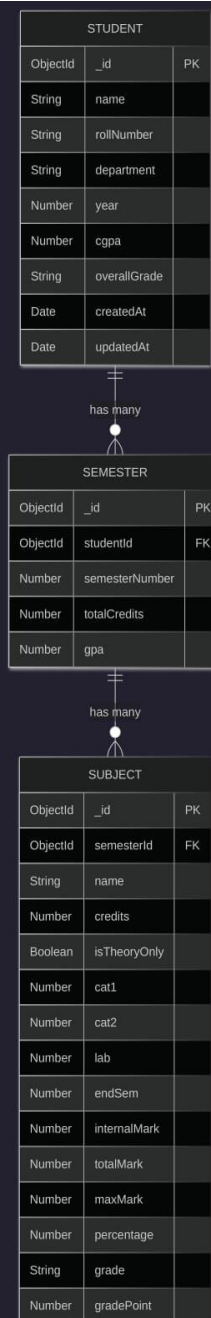
Software Requirements:

- HTML, CSS, JavaScript
- Node.js, Express.js
- MongoDB

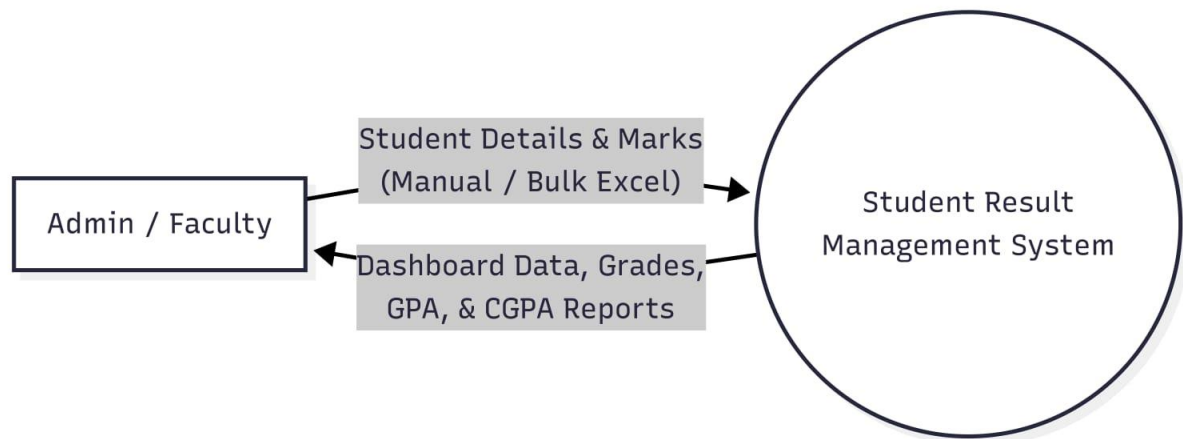
6.USE CASE DIAGRAM



7.ENTITY RELATIONSHIP DIAGRAM



8.DATA FLOW DIAGRAM



9.MODULE DESCRIPTION

Student Module – manages student data

Marks Module – handles marks

Result Module – displays results

Database Module – stores data

10.PROJECT FUNCTIONALITY

- Student Registration
- Marks Entry
- View Records
- Update/Delete
- Result Generation

BACK END CODE

```
const express = require('express');
const router = express.Router();
const Student = require('../models/Student');

// — GET all students —————
router.get('/', async (req, res) => {
  try {
    const students = await Student.find().sort({ createdAt: -1 });
    res.json(students);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// — GET single student —————
router.get('/:id', async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);
    if (!student) return res.status(404).json({ message: 'Student not found' });
    res.json(student);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// — POST create student —————
router.post('/', async (req, res) => {
  try {
    const { name, rollNumber, department, year, semester } = req.body;
    const studentData = { name, rollNumber, department, year, semesters: [] };

    if (semester && semester.semesterNumber && semester.subjects && semester.subjects.length > 0) {
      studentData.semesters.push(semester);
    }

    const student = new Student(studentData);
    const saved = await student.save();
    res.status(201).json(saved);
  } catch (err) {
    if (err.code === 11000) return res.status(400).json({ message: 'Roll number already exists' });
  }
});
```

BACK END CODE

```
    }

    const student = new Student(studentData);
    const saved = await student.save();
    res.status(201).json(saved);
  } catch (err) {
    if (err.code === 11000) return res.status(400).json({ message: 'Roll number already exists' });
    res.status(400).json({ message: err.message });
  }
});

// — PUT update student —————
router.put('/:id', async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);
    if (!student) return res.status(404).json({ message: 'Student not found' });

    const { name, rollNumber, department, year, semester } = req.body;
    if (name) student.name = name;
    if (rollNumber) student.rollNumber = rollNumber;
    if (department) student.department = department;
    if (year) student.year = year;

    if (semester && semester.semesterNumber && semester.subjects && semester.subjects.length > 0) {
      const idx = student.semesters.findIndex(s => s.semesterNumber === semester.semesterNumber);
      if (idx >= 0) {
        student.semesters[idx] = semester; // replace
      } else {
        student.semesters.push(semester); // add
      }
      student.semesters.sort((a, b) => a.semesterNumber - b.semesterNumber);
    }

    const updated = await student.save(); // triggers pre-save calculations
    res.json(updated);
  } catch (err) {
    if (err.code === 11000) return res.status(400).json({ message: 'Roll number already exists' });
    res.status(400).json({ message: err.message });
  }
}
```

MONGODB CONNECTIVITY CODE

```
javascript

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const path = require('path');
require('dotenv').config();

const studentRoutes = require('./routes/students');
const uploadRoutes = require('./routes/upload');

const app = express();
const PORT = process.env.PORT || 3000;

// MongoDB connection
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/studentResultDB';

mongoose.connect(MONGO_URI)
  .then(() => console.log('✅ Connected to MongoDB'))
  .catch(err => console.error('❌ MongoDB connection error:', err));

// Middleware
app.use(cors());
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true }));

// Serve static frontend files
app.use(express.static(path.join(__dirname, 'public')));

// API routes
app.use('/api/students', studentRoutes);
app.use('/api/upload', uploadRoutes);

// Fallback to index.html
app.get('/*splat}', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'index.html'));
});

app.listen(PORT, () => {
  console.log('🚀 Server running at http://localhost:${PORT}');
});
```

FRONT END CODE

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description"
    content="Student Result Management System with GPA, CGPA, CAT marks, Lab and End Sem grading - N">
  <title>Student Result Management System</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&display=swap"
    rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>

<body>

  <!-- ===== HEADER ===== -->
  <header id="app-header">
    <div class="header-inner">
      <div class="logo">
        <span class="logo-icon">🎓</span>
        <h1>Student Result Manager</h1>
      </div>
      <p class="tagline">GPA & CGPA Calculation System – powered by MongoDB</p>
    </div>
  </header>

  <!-- ===== MAIN ===== -->
  <main id="app-main">

    <!-- ——— FORM SECTION ——— -->
    <section class="card form-card" id="form-section">
      <div class="card-header">
        <h2 id="form-title">📝 Add New Student</h2>
        <button class="btn btn-ghost" id="btn-reset-form" title="Reset form" style="display:none;"></button>
      </div>

      <form id="student-form" autocomplete="off">
        <input type="hidden" id="student-id">
```


FRONT END CODE

[illegible]

11.IMPLEMENTATION

Three-tier architecture:

Frontend – HTML, CSS, JS

Backend – Node.js, Express

12.CONCLUSION

The system improves efficiency, reduces errors, and demonstrates modern web development with MongoDB.

REFERENCE

MongoDB Docs

Node.js Docs

Express.js Docs

MDN

W3Schools